MiniJava Compiler – Sprint 1 Guide: Project Setup and Lexer

Author: Deepanshu Chaurasia (Deepu) | **Sprint:** 1 of 4 | **Dates:** 28 Oct to 3 Nov 2026 | **Covers:** FR-1 to FR-10 and FR-43

> **How to use this guide:** Do the steps in order. After every step there is a small check. Type the code yourself instead of only pasting it, because you learn more that way. Do **not** rename any file, package or method. Later sprints depend on these exact names.

1. The V1 Sprint Plan (big picture)

What: V1 is split into 4 sprints of one week each, plus a buffer. **Why:** a small goal every week is easier to finish than one big goal. At the end of every sprint the project builds, all tests pass, and the code is merged to `main`.

Sprint	Dates	Goal	Requirements	Done when
1	28 Oct to 3 Nov	Project setup, Token, Lexer	FR-1 to FR-10, FR-43	`mvn test` is green and `--tokens` prints tokens
2	4 to 10 Nov	AST and Parser	FR-11 to FR-20, FR-41	Source code becomes an AST, syntax errors are reported
3	11 to 17 Nov	Semantic Analyzer, Instructions, Code Generator	FR-21 to FR-29, FR-42	A checked AST becomes a list of instructions
4	18 to 24 Nov	MiniVM, full pipeline, README, release	FR-30 to FR-40, FR-44	`hello.mj` prints 21, 85, 70 and tag `v1.0.0` exists
Buffer	25 to 28 Nov	Bug fixes and polish		V1 released by 28 Nov

Each sprint has its own guide like this one. If you finish a sprint early, tell me and we start the next one early.

2. Sprint 1 Goal

What: A working project and a working lexer. **Why:** the lexer is stage 1. Every later stage needs its `Token`, so we build it first and build it carefully.

Sprint 1 is done when all of these are true:

- [x] ~~The IntelliJ Maven project builds with Java 21~~
- [x] ~~`mvn test` shows **19 tests, 0 failures**~~
- [x] ~~`--tokens examples/hello.mj` prints the correct tokens~~
- [x] ~~`examples/errors/bad_char.mj` gives a clear error message with line and column~~
- [x] ~~Everything is committed to Git, merged to `main`, and tagged `sprint-1`~~

Important rule: every design detail of this sprint was decided *before* writing the code. So all files below stay unchanged in later sprints, except `Main.java`, where we only **add** lines. (V2 will extend the lexer with new tokens, because V2 adds new language features.)

3. Files in this Sprint (where each file goes)

#	File	Step
1	`pom.xml` (project root)	2
2	`.gitignore` (project root)	4
3	`src/main/java/com/deepu/minijava/diagnostics/CompileException.java`	5
4	`src/main/java/com/deepu/minijava/diagnostics/LexerException.java`	5
5	`src/main/java/com/deepu/minijava/lexer/TokenType.java`	6
6	`src/main/java/com/deepu/minijava/lexer/Token.java`	6
7	`src/main/java/com/deepu/minijava/lexer/Lexer.java`	7
8	`src/test/java/com/deepu/minijava/lexer/TokenTest.java`	8

#	File	Step
9	`src/test/java/com/deepu/minijava/lexer/LexerTest.java`	8
10	`src/main/java/com/deepu/minijava/Main.java`	9
11	`examples/hello.mj`	9
12	`examples/arithmetic.mj`	9
13	`examples/errors/bad_char.mj`	9

4. Java Features Used in this Sprint (quick meanings)

Feature	Simple meaning	Where
`enum`	A fixed list of names, like days of the week	`TokenType`
`record`	A short way to write a class that only holds data. Java makes the constructor, getters, `equals` and `toString` for you	`Token`
`switch` with `->`	A cleaner `switch`. No `break` is needed	`Lexer`
`Map.of(...)`	A small read-only dictionary (key and value pairs)	`Lexer` keywords
`RuntimeException`	An error type that you do not have to declare with `throws`	`CompileException`

Step 1: Create the project in IntelliJ

What: An empty Maven project. **Why:** Maven builds the code, runs tests and makes the `.jar` file. **How:**

1. `File > New > Project`.
2. Name: `minijava-compiler`. Build system: **Maven**. JDK: **21** (if it is not in the list, choose *Download JDK* and pick any version 21).
3. Open *Advanced Settings*. GroupId: `com.deepu`. ArtifactId: `minijava-compiler`.
4. Untick *Add sample code* if you see it. Click **Create**.

Check: you see `pom.xml`, `src/main/java` and `src/test/java`. If IntelliJ made a class like `org.example.Main`, delete it.

Step 2: Replace `pom.xml`

What: The project settings file. **Why:** it says which Java version to use, adds JUnit 5, runs the tests, and builds the runnable jar. **How:** select everything in `pom.xml` and replace it with this. Then click the Maven *Reload* button (the small elephant icon with a circular arrow).

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.deepu</groupId>
    <artifactId>minijava-compiler</artifactId>
    <version>1.0-SNAPSHOT</version>
    <name>MiniJava Compiler</name>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <junit.version>5.11.3</junit.version>
    </properties>

    <dependencies>
        <!-- JUnit 5: used only for tests -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <!-- The jar file will be target/minijava-compiler.jar -->
        <finalName>minijava-compiler</finalName>
        <plugins>
```

```
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.2</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.4.2</version>
                <configuration>
                    <archive>
                        <manifest>
                            <mainClass>com.deepu.minijava.Main</mainClass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Important parts:

- `maven.compiler.release 21` makes Maven compile with Java 21.
- `junit-jupiter` with `scope test` is JUnit 5. It is the only dependency, and it is not part of the compiler. We use no ANTLR, no ASM.
- `maven-surefire-plugin` is the plugin that runs your tests when you run `mvn test`.
- `maven-jar-plugin` with `mainClass` is what makes `java -jar minijava-compiler.jar` work later.

Check: Maven finishes loading with no red errors. (If a newer JUnit 5 version exists, you may use it. These versions are known to work together.)

Step 3: Create the packages and folders

What: The folder structure from the design document. **Why:** one package per compiler stage, so the folders tell the story of the pipeline. **How:**

1. Right click `src/main/java` > `New > Package` > `com.deepu.minijava.lexer`. Then create `com.deepu.minijava.diagnostics` the same way.
2. Right click `src/test/java` > `New > Package` > `com.deepu.minijava.lexer`.
3. Right click the project root > `New > Directory` > `examples`. Inside it create `errors`. Also create `docs` and put your downloaded `SRS.md` and `design.md` there.

```
minijava-compiler/
├── pom.xml
├── docs/
├── examples/
│   └── errors/
└── src/
    ├── main/java/com/deepu/minijava/
    │   ├── diagnostics/
    │   └── lexer/
    └── test/java/com/deepu/minijava/
        └── lexer/
```

Step 4: Git setup

What: Start version control. **Why:** Git is your safety net. If something breaks, you can go back. It is also what recruiters see.

Create `.gitignore` in the project root:

```
target/
.idea/
*.iml
out/
```

Why these lines: `target/` holds build output, and `.idea/` and `*.iml` are IntelliJ settings. We never commit them.

In the IntelliJ terminal (create an empty repository named `minijava-compiler` on GitHub first, without README):

```
git init
git add .
git commit -m "chore: initial project setup"
git branch -M main
git remote add origin https://github.com/<your-username>/minijava-
compiler.git
git push -u origin main
git checkout -b feature/sprint-1-lexer
```

Check: your GitHub page shows the project. You are now on the branch `feature/sprint-1-lexer`. All Sprint 1 work happens on this branch. `main` always stays clean.

Step 5: Error classes (diagnostics package)

What: The error types of the compiler. **Why:** every stage must report errors in the same style, with a line and a column. We create the base class now, so the parser and semantic analyzer can reuse it later.

Create `diagnostics/CompileException.java`:

```
package com.deepu.minijava.diagnostics;

/**
 * Base class for every error found while compiling a MiniJava program
 * (lexical, syntax and semantic errors).
 * It extends RuntimeException, so we do not need "throws" everywhere.
 */
public abstract class CompileException extends RuntimeException {

    private final String stage;   // "Lexical", "Syntax" or "Semantic"
    private final int line;
    private final int column;

    protected CompileException(String stage, String message, int line, int
column) {
        super(message);
        this.stage = stage;
        this.line = line;
        this.column = column;
    }

    public String getStage() {
```

```
        return stage;
    }

    public int getLine() {
        return line;
    }

    public int getColumn() {
        return column;
    }

    /** Text shown to the user. Example: [Lexical Error] line 1, column 11:
Unexpected character '@' */
    public String format() {
        return "[" + stage + " Error] line " + line + ", column " + column +
": " + getMessage();
    }
}
```

Create `diagnostics/LexerException.java`:

```
package com.deepu.minijava.diagnostics;

/** Thrown by the Lexer when the source text contains something that is not
valid MiniJava. */
public class LexerException extends CompileException {

    public LexerException(String message, int line, int column) {
        super("Lexical", message, line, column);
    }
}
```

Important parts:

- `abstract` means nobody can create a plain `CompileException`. They must use a child class that names the stage.
- `format()` makes every error look the same: `[Lexical Error] line 1, column 11: ...`. In Sprint 2 and 3 we add `ParseException` (stage "Syntax") and `SemanticException` (stage "Semantic") the same way.

Check: no red lines in IntelliJ.

Step 6: TokenType and Token

What: The vocabulary of the language. **Why:** the parser should not care about spaces, comments or single characters. It wants clean pieces like "keyword int", "name age", "number 21".

Create `lexer/TokenType.java`:

```
package com.deepu.minijava.lexer;

/** Every kind of token that MiniJava V1 knows. */
public enum TokenType {
    // Keywords (IF, ELSE and WHILE are reserved now and used from V2)
    INT, PRINT, IF, ELSE, WHILE,

    // Literals and names
    INTEGER, IDENTIFIER,

    // Operators
    PLUS, MINUS, STAR, SLASH, EQUAL,

    // Symbols
    LEFT_PAREN, RIGHT_PAREN, LEFT_BRACE, RIGHT_BRACE, SEMICOLON,

    // Special: marks the end of the file
    EOF
}
```

Create `lexer/Token.java`:

```
package com.deepu.minijava.lexer;

/**
 * One small meaningful piece of the source code.
 *
 * @param type   what kind of token it is
 * @param lexeme the exact text from the file, for example "age" or "21"
 * @param line   line number where the token starts (first line is 1)
 * @param column column number where the token starts (first column is 1)
 */
public record Token(TokenType type, String lexeme, int line, int column) {

    /** Column just after the last character of this token. Used for error
```

```
positions. */
    public int endColumn() {
        return column + lexeme.length();
    }

    @Override
    public String toString() {
        return type + "(\"" + lexeme + "\") " + line + ":" + column;
    }
}
```

Important parts:

- A token stores `line` and `column` from the very start. This is why every later error can say *where* the problem is.
- `endColumn()` is used by the parser in Sprint 2. For a missing `;` we point to the place just *after* the previous token, so the error shows on the correct line.
- `toString()` prints tokens like `INTEGER("21") 3:11`. That is the format `--tokens` will show.

Check: no red lines.

Step 7: The Lexer

What: The program that reads source text and gives a list of tokens. **Why:** it is stage 1 of the compiler. **Analogy:** reading a sentence and cutting it into words. It reads one character at a time, decides what kind of word starts there, reads the whole word, and moves on.

Create `lexer/Lexer.java`:

```
package com.deepu.minijava.lexer;

import com.deepu.minijava.diagnostics.LexerException;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * Stage 1 of the compiler: reads the source text character by character
 * and turns it into a list of tokens.
 * Use one Lexer object for one source text and call scanTokens() only once.
```

```
 */
public class Lexer {

    private static final Map<String, TokenType> KEYWORDS = Map.of(
            "int", TokenType.INT,
            "print", TokenType.PRINT,
            "if", TokenType.IF,
            "else", TokenType.ELSE,
            "while", TokenType.WHILE
    );

    private final String source;
    private final List<Token> tokens = new ArrayList<>();

    private int start = 0;        // index where the current token started
    private int current = 0;      // index of the next character to read
    private int line = 1;         // line of the next character to read
    private int column = 1;       // column of the next character to read
    private int startLine = 1;    // line where the current token started
    private int startColumn = 1;  // column where the current token started

    public Lexer(String source) {
        this.source = source;
    }

    /** Reads the whole source text. The last token is always EOF. */
    public List<Token> scanTokens() {
        while (!isAtEnd()) {
            start = current;
            startLine = line;
            startColumn = column;
            scanToken();
        }
        tokens.add(new Token(TokenType.EOF, "", line, column));
        return tokens;
    }

    /** Reads one token (or skips whitespace and comments). */
    private void scanToken() {
        char c = advance();
        switch (c) {
            case ' ', '\t', '\r', '\n' -> { }   // whitespace: ignore it
            case '(' -> addToken(TokenType.LEFT_PAREN);
```

```
            case ')' -> addToken(TokenType.RIGHT_PAREN);
            case '{' -> addToken(TokenType.LEFT_BRACE);
            case '}' -> addToken(TokenType.RIGHT_BRACE);
            case ';' -> addToken(TokenType.SEMICOLON);
            case '+' -> addToken(TokenType.PLUS);
            case '-' -> addToken(TokenType.MINUS);
            case '*' -> addToken(TokenType.STAR);
            case '=' -> addToken(TokenType.EQUAL);
            case '/' -> {
                if (match('/')) {
                    // A comment: skip everything until the end of the line.
                    while (!isAtEnd() && peek() != '\n') {
                        advance();
                    }
                } else {
                    addToken(TokenType.SLASH);
                }
            }
            default -> {
                if (isDigit(c)) {
                    number();
                } else if (isIdentifierStart(c)) {
                    identifier();
                } else {
                    throw new LexerException("Unexpected character '" + c +
"'", startLine, startColumn);
                }
            }
        }
    }

    /** Reads a whole integer literal such as 21. */
    private void number() {
        while (isDigit(peek())) {
            advance();
        }
        // A number must not touch letters or '_' (for example 12abc).
        if (isIdentifierStart(peek())) {
            while (isIdentifierPart(peek())) {
                advance();
            }
            throw new LexerException("Invalid number '" +
source.substring(start, current) + "'", startLine, startColumn);
```

```
        }
        String text = source.substring(start, current);
        try {
            Integer.parseInt(text);   // only to check that it fits in a 32-
bit int
        } catch (NumberFormatException e) {
            throw new LexerException("Integer literal '" + text + "' is too
large (maximum is "
                    + Integer.MAX_VALUE + ")", startLine, startColumn);
        }
        addToken(TokenType.INTEGER);
    }

    /** Reads a whole word. It is a keyword (like int) or an identifier (like
age). */
    private void identifier() {
        while (isIdentifierPart(peek())) {
            advance();
        }
        String text = source.substring(start, current);
        addToken(KEYWORDS.getOrDefault(text, TokenType.IDENTIFIER));
    }

    private void addToken(TokenType type) {
        String lexeme = source.substring(start, current);
        tokens.add(new Token(type, lexeme, startLine, startColumn));
    }

    /** Moves forward by one character and keeps line and column up to date.
*/
    private char advance() {
        char c = source.charAt(current++);
        if (c == '\n') {
            line++;
            column = 1;
        } else {
            column++;
        }
        return c;
    }

    /** Looks at the next character without moving. Returns '\0' at the end.
*/
```

```
    private char peek() {
        return isAtEnd() ? '\0' : source.charAt(current);
    }

    /** Moves forward only if the next character is the expected one. */
    private boolean match(char expected) {
        if (isAtEnd() || source.charAt(current) != expected) {
            return false;
        }
        advance();
        return true;
    }

    private boolean isAtEnd() {
        return current >= source.length();
    }

    private static boolean isDigit(char c) {
        return c >= '0' && c <= '9';
    }

    private static boolean isIdentifierStart(char c) {
        return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_';
    }

    private static boolean isIdentifierPart(char c) {
        return isIdentifierStart(c) || isDigit(c);
    }
}
```

Important parts (the idea of the lexer):

Part	What it does and why
`start` and `current`	`start` is where the current token began. `current` is the next character to read. The token text is `source.substring(start, current)`
`advance()`	Reads one character and moves forward. It also keeps `line` and `column` correct, so every token knows its position

Part	What it does and why
`peek()`	Looks at the next character *without* moving. This is how we know where a number or a word ends
`match('/')`	If the next character is `/`, take it. Two slashes mean a comment. One slash means division
`startLine`, `startColumn`	We save the position *before* reading the token, so the token points to its first character
`KEYWORDS` map	After reading a whole word, we look it up. Found, so it is a keyword. Not found, so it is an identifier

Rules fixed now (so they never change later):

- Only the English letters `a` to `z`, `A` to `Z` and `_` can start or continue a name. Keywords are case sensitive (`Int` is a normal name).
- Only `//` comments exist in V1.
- A tab counts as one column. `\r` is treated as whitespace, so Windows files work.
- Numbers are decimal and must fit in a 32-bit `int` (maximum 2147483647). `12abc` and too big numbers are errors.
- The lexer stops at the first error.

Example trace for `int x = 5;`

Character	What happens	Token made
`i`, `n`, `t`	A word starts. Read all letters. It is in `KEYWORDS`	`INT("int") 1:1`
space	Ignored	
`x`	A word. Not in `KEYWORDS`	`IDENTIFIER("x") 1:5`
`=`	Single character token	`EQUAL("=") 1:7`
`5`	A number. Read all digits	`INTEGER("5") 1:9`
`;`	Single character token	`SEMICOLON(";") 1:10`
end of text	Always add EOF	`EOF("") 1:11`

Check: no red lines. Build the project (`Build > Build Project`).

Step 8: Tests

What: JUnit tests for the token and the lexer. **Why:** tests prove each requirement works, and they protect you when you change code later. A real team never says "it works" without tests.

Create `src/test/java/com/deepu/minijava/lexer/TokenTest.java`:

```
package com.deepu.minijava.lexer;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class TokenTest {

    @Test
    void endColumnIsJustAfterTheLastCharacter() {
        Token token = new Token(TokenType.IDENTIFIER, "age", 1, 5);
        assertEquals(8, token.endColumn());
    }

    @Test
    void toStringIsReadable() {
        Token token = new Token(TokenType.INTEGER, "21", 3, 11);
        assertEquals("INTEGER(\"21\") 3:11", token.toString());
    }
}
```

Create `src/test/java/com/deepu/minijava/lexer/LexerTest.java`:

```
package com.deepu.minijava.lexer;

import com.deepu.minijava.diagnostics.LexerException;
import org.junit.jupiter.api.Test;

import java.util.List;

import static com.deepu.minijava.lexer.TokenType.*;
import static org.junit.jupiter.api.Assertions.*;

class LexerTest {
```

```
    private List<Token> scan(String source) {
        return new Lexer(source).scanTokens();
    }

    private List<TokenType> types(String source) {
        return scan(source).stream().map(Token::type).toList();
    }

    @Test
    void emptySourceGivesOnlyEof() {
        List<Token> tokens = scan("");
        assertEquals(1, tokens.size());
        assertEquals(EOF, tokens.get(0).type());
    }

    @Test
    void variableDeclaration() {
        List<Token> tokens = scan("int age = 21;");
        assertEquals(List.of(INT, IDENTIFIER, EQUAL, INTEGER, SEMICOLON,
EOF),
                tokens.stream().map(Token::type).toList());
        assertEquals("age", tokens.get(1).lexeme());
        assertEquals("21", tokens.get(3).lexeme());
    }

    @Test
    void keywordsAreNotIdentifiers() {
        assertEquals(List.of(INT, IDENTIFIER, PRINT, IDENTIFIER, IF, ELSE,
WHILE, EOF),
                types("int integer print printer if else while"));
    }

    @Test
    void operatorsAndSymbols() {
        assertEquals(List.of(PLUS, MINUS, STAR, SLASH, EQUAL, LEFT_PAREN,
RIGHT_PAREN,
                        LEFT_BRACE, RIGHT_BRACE, SEMICOLON, EOF),
                types("+ - * / = ( ) { } ;"));
    }

    @Test
    void lineCommentIsSkipped() {
        List<Token> tokens = scan("// hello\nint x;");
```

```
        assertEquals(List.of(INT, IDENTIFIER, SEMICOLON, EOF),
                tokens.stream().map(Token::type).toList());
        assertEquals(2, tokens.get(0).line());
    }

    @Test
    void commentAtEndOfFileWithoutNewline() {
        assertEquals(List.of(INT, IDENTIFIER, SEMICOLON, EOF), types("int x;
// the end"));
    }

    @Test
    void singleSlashIsDivision() {
        assertEquals(List.of(INTEGER, SLASH, INTEGER, EOF), types("10 / 2"));
    }

    @Test
    void positionsOnOneLine() {
        List<Token> tokens = scan("int age = 21;");
        int[] expectedColumns = {1, 5, 9, 11, 13, 14};
        for (int i = 0; i < expectedColumns.length; i++) {
            assertEquals(1, tokens.get(i).line());
            assertEquals(expectedColumns[i], tokens.get(i).column());
        }
    }

    @Test
    void positionsAcrossLines() {
        List<Token> tokens = scan("int a;\n  int b;");
        Token secondInt = tokens.get(3);
        assertEquals(INT, secondInt.type());
        assertEquals(2, secondInt.line());
        assertEquals(3, secondInt.column());
        assertEquals(7, tokens.get(4).column());   // the name b
    }

    @Test
    void windowsLineEndings() {
        List<Token> tokens = scan("int a;\r\nint b;");
        assertEquals(2, tokens.get(3).line());
        assertEquals(1, tokens.get(3).column());
    }
```

```
    @Test
    void identifiersWithDigitsAndUnderscore() {
        List<Token> tokens = scan("total_1 _x x2");
        assertEquals(List.of(IDENTIFIER, IDENTIFIER, IDENTIFIER, EOF),
                tokens.stream().map(Token::type).toList());
        assertEquals("total_1", tokens.get(0).lexeme());
        assertEquals("_x", tokens.get(1).lexeme());
        assertEquals("x2", tokens.get(2).lexeme());
    }

    @Test
    void largestIntIsAccepted() {
        List<Token> tokens = scan("2147483647");
        assertEquals(INTEGER, tokens.get(0).type());
        assertEquals("2147483647", tokens.get(0).lexeme());
    }

    @Test
    void invalidCharacter() {
        LexerException e = assertThrows(LexerException.class, () -> scan("int
x = 5 @ 3;"));
        assertEquals("Unexpected character '@'", e.getMessage());
        assertEquals(1, e.getLine());
        assertEquals(11, e.getColumn());
    }

    @Test
    void invalidCharacterOnSecondLine() {
        LexerException e = assertThrows(LexerException.class, () -> scan("int
a;\n@"));
        assertEquals(2, e.getLine());
        assertEquals(1, e.getColumn());
    }

    @Test
    void numberFollowedByLettersIsAnError() {
        LexerException e = assertThrows(LexerException.class, () ->
scan("12abc"));
        assertEquals("Invalid number '12abc'", e.getMessage());
        assertEquals(1, e.getColumn());
    }

    @Test
```

```
    void integerTooLargeIsAnError() {
        LexerException e = assertThrows(LexerException.class, () ->
scan("99999999999"));
        assertTrue(e.getMessage().contains("too large"));
    }

    @Test
    void printStatement() {
        assertEquals(List.of(PRINT, LEFT_PAREN, INTEGER, PLUS, INTEGER, STAR,
INTEGER,
                             RIGHT_PAREN, SEMICOLON, EOF),
                types("print(2 + 3 * 4);"));
    }
}
```

Important parts:

- `@Test` marks a method as a test. `assertEquals(expected, actual)` fails the test when the two values are different. `assertThrows` checks that the right error is thrown.
- The tests cover the SRS list for the lexer: correct tokens, keywords, comments, positions, invalid characters.
- The helper `scan(...)` keeps each test short.

Check: run the tests (Step 10). You should see 19 tests pass: 2 in `TokenTest` and 17 in `LexerTest`.

Step 9: Main and example files

What: The entry point of the compiler, and sample programs. **Why:** `Main` is how users run the compiler. In Sprint 1 it only runs the lexer, but its shape is already final. In later sprints we **add** the parser, analyzer, generator and VM lines. We do not rewrite it.

Create `src/main/java/com/deepu/minijava/Main.java`:

```
package com.deepu.minijava;

import com.deepu.minijava.diagnostics.CompileException;
import com.deepu.minijava.lexer.Lexer;
import com.deepu.minijava.lexer.Token;

import java.io.IOException;
```

```
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

/** Entry point: Source File -> Lexer -> (more stages are added in later
sprints) */
public class Main {

    private static final int EXIT_OK = 0;
    private static final int EXIT_ERROR = 1;   // compile error or runtime
error in the .mj program
    private static final int EXIT_USAGE = 2;   // wrong command line or
unreadable file

    public static void main(String[] args) {
        System.exit(run(args, System.out, System.err));
    }

    /** Does all the work and returns the exit code. Kept separate from
main() so tests can call it. */
    static int run(String[] args, PrintStream out, PrintStream err) {
        boolean showTokens = false;
        String fileName = null;

        for (String arg : args) {
            if (arg.equals("--tokens")) {
                showTokens = true;
            } else if (arg.startsWith("--")) {
                err.println("Unknown option: " + arg);
                return usage(err);
            } else if (fileName == null) {
                fileName = arg;
            } else {
                err.println("Only one source file is allowed.");
                return usage(err);
            }
        }

        if (fileName == null) {
            return usage(err);
        }
```

```
        if (!fileName.endsWith(".mj")) {
            err.println("The source file must have the .mj extension: " +
fileName);
            return EXIT_USAGE;
        }

        String source;
        try {
            source = readSource(Path.of(fileName));
        } catch (IOException e) {
            err.println("Cannot read file '" + fileName + "': " +
e.getMessage());
            return EXIT_USAGE;
        }

        try {
            List<Token> tokens = new Lexer(source).scanTokens();

            if (showTokens) {                       // this option stays for
the whole project
                for (Token token : tokens) {
                    out.println(token);
                }
                return EXIT_OK;
            }

            // SPRINT 2: the Parser is added here.
            // SPRINT 3: the SemanticAnalyzer and CodeGenerator are added
here.
            // SPRINT 4: MiniVM is added here (and its VmException is caught
below).
            out.println("Lexing finished: " + tokens.size() + " tokens. (The
parser is not built yet.)");
            return EXIT_OK;
        } catch (CompileException e) {
            err.println(e.format());
            return EXIT_ERROR;
        }
    }

    private static String readSource(Path path) throws IOException {
        String text = Files.readString(path, StandardCharsets.UTF_8);
        if (text.startsWith("\uFEFF")) {            // remove the invisible
```

```
BOM some Windows editors add
            text = text.substring(1);
        }
        return text;
    }

    private static int usage(PrintStream err) {
        err.println("Usage: java -jar minijava-compiler.jar [--tokens]
<file.mj>");
        return EXIT_USAGE;
    }
}
```

Important parts:

- **Exit codes:** `0` means success, `1` means an error in your MiniJava program, `2` means wrong usage or unreadable file. Scripts and CI use these numbers.
- `run(...)` is separate from `main(...)`. `main` calls `System.exit`, which would stop a test. `run` just returns the code, so the end-to-end test in Sprint 4 can call it.
- `catch (CompileException e)` prints the nice message from `format()`. This one catch will handle lexer, parser and semantic errors, because they all extend `CompileException`.
- `--tokens` prints the tokens and stops. It stays in the project as a debugging tool.

Create `examples/hello.mj`:

```
// hello.mj

int age = 21;
int marks = 85;

print(age);
print(marks);
print(10 + 20 * 3);
```

Create `examples/arithmetic.mj`:

```
print(2 + 3 * 4);
print((2 + 3) * 4);
print(10 - 4 - 3);
print(-5 + 8);
```

Create `examples/errors/bad_char.mj`:

```
int x = 5 @ 3;
```

(IntelliJ may not know the `.mj` type. If it asks, choose *Plain text*.)

Step 10: Run and verify

1. Run the tests. In the IntelliJ Maven panel (right side): `Lifecycle > test`. Or in the terminal: `mvn test`.

Expected at the end:

```
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

2. Run `Main` from IntelliJ. Open `Main.java`, click the green arrow next to `main`, then `Edit Configurations`. Set *Program arguments* to `--tokens examples/hello.mj`. Working directory must be the project root. Run it.

Expected output (first lines):

```
INT("int") 3:1
IDENTIFIER("age") 3:5
EQUAL("=") 3:9
INTEGER("21") 3:11
SEMICOLON(";") 3:13
INT("int") 4:1
IDENTIFIER("marks") 4:5
EQUAL("=") 4:11
INTEGER("85") 4:13
SEMICOLON(";") 4:15
PRINT("print") 6:1
...
PRINT("print") 8:1
LEFT_PAREN("(") 8:6
INTEGER("10") 8:7
PLUS("+") 8:10
INTEGER("20") 8:12
STAR("*") 8:15
```

```
INTEGER("3") 8:17
RIGHT_PAREN(")") 8:18
SEMICOLON(";") 8:19
EOF("") 9:1
```

The comment line and blank lines are skipped, and the numbers `3:1` show the line and column. The last line is `EOF("") 9:1` if your file ends with a new line. It is `8:20` if it does not.

3. Run without `--tokens`. Change the program arguments to `examples/hello.mj`. Expected:

```
Lexing finished: 30 tokens. (The parser is not built yet.)
```

4. Run the error example. Program arguments: `examples/errors/bad_char.mj`. Expected on the error stream (red text in IntelliJ):

```
[Lexical Error] line 1, column 11: Unexpected character '@'
```

5. Optional: build the jar. In the terminal: `mvn clean package`, then:

```
java -jar target/minijava-compiler.jar --tokens examples/hello.mj
java -jar target/minijava-compiler.jar examples/errors/bad_char.mj
```

Check the exit code after the second command. Windows PowerShell: `$LASTEXITCODE`. Windows cmd: `echo %errorlevel%`. Linux and macOS: `echo $?`. It should be `1`. Running the jar with no file should print the usage text and give exit code `2`.

Step 11: Commit, merge and tag

Why: a sprint is only finished when the work is safely in `main`. Make small commits as you go, one per step if you can:

After step	Commit message
5	`feat: add CompileException and LexerException`
6	`feat: add TokenType and Token`
7	`feat: add Lexer`

After step	Commit message
8	`test: add Token and Lexer tests`
9	`feat: add Main with --tokens option and examples`

When all checks pass:

```
git push -u origin feature/sprint-1-lexer
```

On GitHub, open a **Pull Request** from `feature/sprint-1-lexer` into `main`, read your own changes once, then merge it. Back in the terminal:

```
git checkout main
git pull
git tag sprint-1
git push origin sprint-1
```

5. Common Problems

Problem	Fix
`release version 21 not supported`	`File > Project Structure > Project`: set SDK to 21. Then `Settings > Build > Build Tools > Maven > Runner`: set JRE to 21
Red `import` or `package does not exist`	Reload Maven (elephant icon). Check that the folders are exactly as in Step 3
`Tests run: 0`	Test class must be in `src/test/java` and its name must end with `Test`. Also check that `junit-jupiter` is in `pom.xml`
`mvn` is not recognized in the terminal	Use the IntelliJ Maven panel instead, or install Maven
Token positions are different from the guide	Check that you copied `advance()` exactly, and that your file has the same lines as `hello.mj`

Problem	Fix
A test fails	Read the message: `expected: <X> but was: <Y>`. Tell me the test name and the message, and we find the cause together

6. Sprint Review Checklist

Before you tell me "Sprint 1 done", confirm:

- [x] ~~`mvn test` shows 19 passed tests~~
- [x] ~~`--tokens` output for `hello.mj` matches this guide~~
- [x] ~~The `bad_char.mj` error message matches this guide~~
- [ ] I can explain in my own words what `start`, `current`, `peek()` and `advance()` do
- [x] ~~I can explain why a token stores line and column~~
- [x] ~~Code is merged to `main` and tagged `sprint-1`~~

Next: Sprint 2 builds the AST classes and the recursive-descent parser. It uses `Token` and `Lexer` exactly as you wrote them today.